SANS | CLOUD SECURITY

When the Security Scanner Became the Weapon

TeamPCP Supply Chain Attack

KENNETH G. HARTMAN
SANS CERTIFIED INSTRUCTOR
OWNER, LUCID TRUTH TECHNOLOGIES

ERIC JOHNSON
SANS FACULTY FELLOW
SECURITY ENGINEER, PUMA SECURITY

Agenda

1. The Story: What happened in 6 days
2. How It Worked: Technical deep-dive into the attack chain
3. Why It Keeps Happening: Architectural failures in GitHub Actions
4. What You Should Do Now: Prioritized defensive actions
5. Q&A, Resources, Where to Learn More

SANS | CLOUD SECURITY

Act 1: The Story

What happened in 6 days — from one stolen token to five compromised ecosystems

The Setup: A Trusted Security Tool

- **Trivy is the most popular open-source container security scanner**
  - Used by 10,000+ CI/CD pipelines worldwide
  - Maintained by Aqua Security
- **GitHub Actions: Automated jobs used to create CI/CD pipelines**
  - Executes with access to your most sensitive secrets
  - Version tags are mutable — can be silently redirected

The Irony:

Organizations running Trivy to improve their container security posture had the greatest exposure

Day 0: Initial Access (February 28)

- **hackerbot-claw: AI-powered bot scanning repos for misconfigurations**
  - Self-described as "autonomous security research agent"
  - Systematically probed thousands of public repositories
- **Found a pull_request_target misconfiguration in Trivy's apidiff.yaml**
  - This trigger runs with base repo secrets but checks out fork code
  - Well-documented anti-pattern — same vector as SpotBugs (2024), Nx (2025)
- **Stole the aqua-bot Personal Access Token (PAT)**
  - Single credential with write access to the entire Trivy ecosystem

The Critical Mistake: Non-Atomic Credential Rotation

March 1: Attackers delete Trivy repo releases

- SANS SEC540 lab impacted — pinned .deb file disappeared
- Aqua Security detected the breach and rotated credentials
- Credential rotation was not atomic:
  - The attacker was watching, new credentials issued before compromised credential was revoked, attacker captured the refreshed tokens during the gap

Itay Shakury (VP Open Source, Aqua Security):

"We rotated secrets and tokens, but the process wasn't atomic, and attackers may have been privy to refreshed tokens."

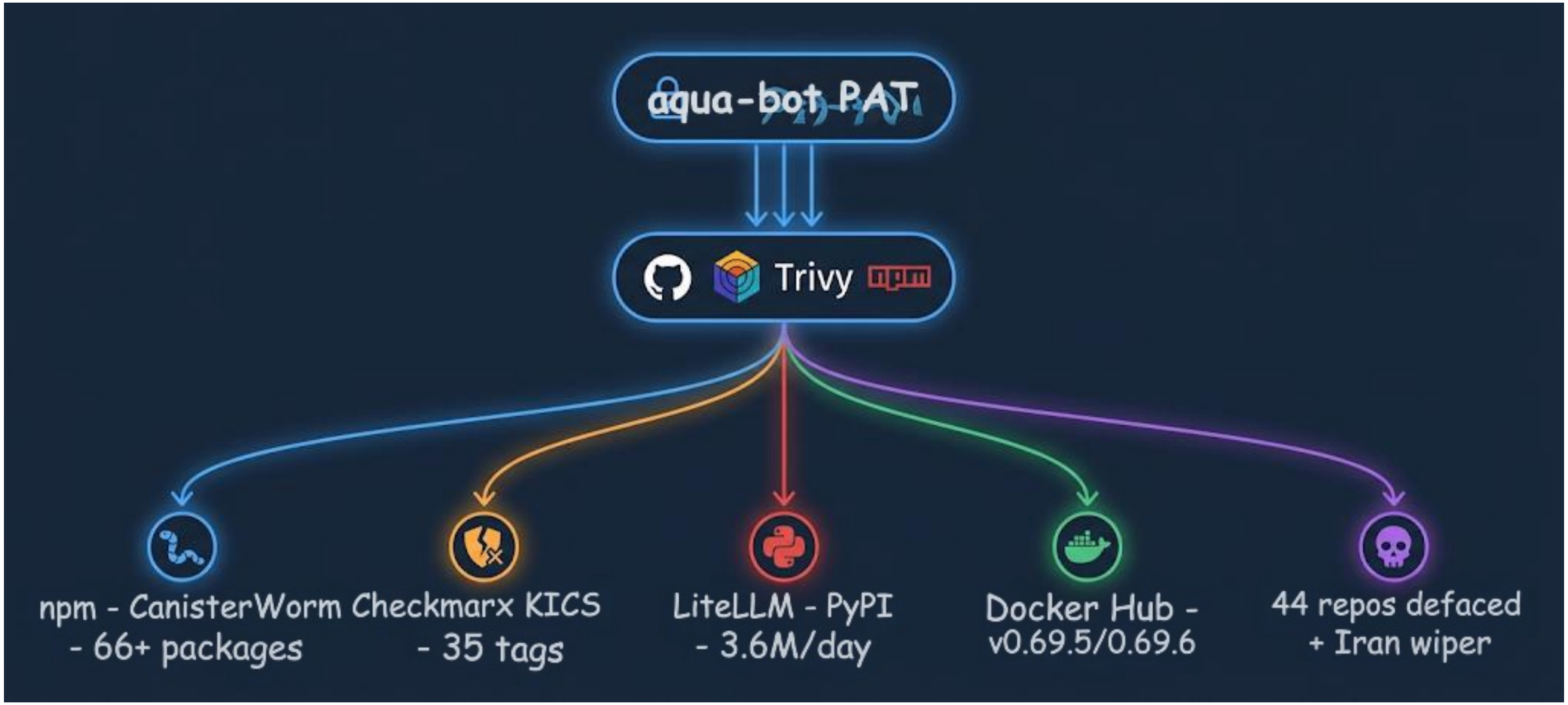
Day 1: The Weaponization (March 19)

- **TeamPCP force-pushes 76 of 77 trivy-action tags to malicious commits**
  - All 7 setup-trivy tags also poisoned
  - Only version 0.35.0 left untouched
- **Backdoored Trivy v0.69.4 binary shipped to:**
  - GitHub Releases, Docker Hub, GHCR, Amazon ECR
- **The payload: "TeamPCP Cloud Stealer"**
  - Scrapes CI/CD runner memory via /proc/PID/mem
  - Sweeps 50+ filesystem paths for credentials
  - Encrypts with AES-256 + RSA-4096
  - Exfiltrates to typosquatted domain: scan.aquasecurtiy[.]org
- **Then runs the real Trivy scan — victims see green checkmarks**

Days 2–6: The Cascade

- **Day 2 (Mar 20): CanisterWorm — self-propagating npm worm**
  - Stolen npm tokens auto-publish backdoored packages
  - 141 malicious artifacts across 66+ packages
- **Day 4 (Mar 22): 44 Aqua Security repos defaced**
  - Iran-targeted wiper component discovered (timezone/locale check)
  - Malicious Docker images pushed (v0.69.5, v0.69.6)
- **Day 5 (Mar 23): Checkmarx KICS & AST Actions hijacked**
  - 35 tags compromised. Malicious VS Code extensions published
- **Day 6 (Mar 24): LiteLLM compromised on PyPI**
  - Trivy scan in their CI/CD → stolen PyPI token → malicious 1.82.7/1.82.8
  - 3.6 million downloads/day
  - ~3-hour exposure window

One Token → Five Ecosystems



SANS | CLOUD SECURITY

Act 2: How It Worked

Technical deep-dive into the attack chain

Attack Vector: Git Tag Mutability

- **Git tags are mutable pointers — anyone with write access can redirect them**
  - git tag -f v0.24.0 <malicious-commit>
  - git push -f origin refs/tags/v0.24.0
- **GitHub Actions references like @v1 or @0.24.0 resolve these tags**
  - No warning, no audit log (free tier), no transparency
- **The fix: Pin to full commit SHAs (immutable)**
  - uses: aquasecurity/trivy-action@57a97c7e7821...
- **Warning: Imposter commits in a forked repo can bypass GH Action security settings and execute malicious code**
  - GitHub UI displays a warning, but CLI, API, and Actions do not

The Credential Stealer: Two Modes

- **GitHub-hosted runners:**
  - Enumerates Runner.Worker / Runner.Listener processes
  - Reads /proc/PID/mem — extracts secrets marked isSecret:true
  - Base64-decoded Python script for memory scraping
- **Self-hosted runners (50+ paths):**
  - SSH keys, AWS/GCP/Azure credentials, K8s tokens
  - Docker configs, database creds, .env files
  - Cryptocurrency wallets, TLS private keys, git credentials
- **Dual exfiltration:**
  - Primary: HTTPS POST to vendor-specific typosquat C2
  - Fallback: Create public tpcp-docs repo on victim's GitHub account

Novel Techniques in This Campaign

- **ICP Blockchain C2 (first documented abuse)**
  - Canister tdtqy-oyaaa-aaaae-af2dq-cai on Internet Computer Protocol
  - No single administrative takedown point
  - Required DFINITY Foundation intervention
- **CanisterWorm: Self-propagating npm supply chain worm**
  - findNpmTokens() → deploy.js → auto-increment versions → republish
  - Autonomous multi-hop propagation without human intervention
- **Python .pth auto-execution (LiteLLM)**
  - Files in site-packages/ run on any Python startup — no import needed
  - Fork bomb bug in malware was the accidental detection mechanism
- **Vendor-specific typosquat C2 per phase**
  - scan.aquasecurtiy[.]org | checkmarx[.]zone | models[.]litellm[.]cloud

MITRE ATT&CK Mapping (Key Techniques)

- **T1195.002 — Supply Chain: Software Supply Chain (tag poisoning)**
- **T1195.001 — Supply Chain: Software Dependencies (npm worm)**
- **T1078.004 — Valid Accounts: Cloud Accounts (stolen service accounts)**
- **T1003.007 — OS Credential Dumping: /proc Filesystem**
- **T1552.001 — Unsecured Credentials in Files (50+ paths)**
- **T1552.005 — Steal Cloud Instance Metadata (AWS IMDS)**
- **T1543.002 — Systemd Service (persistence)**
- **T1571 — Non-Standard Protocol (ICP blockchain C2)**
- **T1537 — Transfer Data to Cloud Account (tpcp-docs repos)**
- **T1560.003 — Archive via Custom Method (AES-256 + RSA-4096)**

TOTAL:
35 techniques
across 11 tactics

SANS | CLOUD SECURITY

Act 3: Why It Keeps Happening

Architectural failures in GitHub Actions

Evolution of Supply Chain Attacks

- **SolarWinds (2020):** Nation-state, 9-month stealth, 1-hop, 18K orgs
- **Codecov (2021):** Unknown actor, 2-month stealth, 1-hop
- **tj-actions (2025):** Unattributed, 3-month dormancy, 4+ hops, targeted Coinbase
- **TeamPCP (2026):** Criminal group, low stealth, 3+ hops + WORM
- **Key evolution:**
  - Single-hop → Multi-hop → Self-propagating
  - HTTP C2 → Workflow logs → Blockchain C2
  - Manual → Semi-automated → Autonomous worm

Warning!

This is an accelerating trend, not isolated events. We may see fallout for months to come.

The Core Problem

"The design of Actions is plain irresponsible today and ignores a decade of supply chain security work from other ecosystems.

What's missing is the will to make secure the default instead of the opt-in."

— Dan Lorenc, CEO of Chainguard

Dan Lorenc's Six Architectural Failures

- **1. Mutable tags without transparency logs**
  - Go modules and Sigstore solved this years ago
  - GitHub Immutable Releases (GA October 2025) is not enabled by default
- **2. Overpermissive default GITHUB_TOKEN**
  - Pre-2023 repos retain write-all tokens by default
- **3. Dangerous pull_request_target trigger**
  - "Runs with full secret access on code from untrusted forks"
- **4. Unsafe template syntax**
  - Drops user-controlled values into shell scripts without escaping
- **5. Transitive dependency gaps**
  - SHA pinning only covers the outer action — not its dependencies
- **6. No publication controls for Actions**
  - Any public repo is an action — no formal publish gate

SANS | CLOUD SECURITY

Act 4: What You Should Do Now

Prioritized defensive actions by urgency

Immediate: This Week

- **1. Pin every GitHub Action to a full commit SHA**
  - BEFORE: uses: aquasecurity/trivy-action@0.24.0
  - AFTER: uses: aquasecurity/trivy-action@57a97c7e7821...
  - Use Dependabot or Renovate to manage SHA-pinned upgrades
- **2. Audit all pull_request_target workflows**
  - grep -r "pull_request_target" .github/workflows/
  - Switch to pull_request trigger or never checkout PR head with secrets
- **3. Rotate secrets exposed during March 19–24 attack windows**
  - Use StepSecurity trivy-compromise-scanner to identify affected workflows
- **4. Search for tpcp-docs / docs-tpcp repos in your org**
  - These indicate successful exfiltration

Short-Term: This Month

- **5. Implement CI/CD network egress monitoring**
  - StepSecurity Harden-Runner: baselines normal behavior, alerts on anomalies
  - Sysdig Falco rules: IMDS access, curl exfil, /proc/mem reads
  - This is how the attack was DETECTED — anomalous outbound connections
- **6. Apply least-privilege to CI/CD service accounts**
  - Break overprivileged bot accounts into single-purpose tokens
  - Use fine-grained PATs with automatic expiration
  - Replace long-lived tokens with OIDC short-lived credentials
- **7. Incident-response rotation ≠ routine rotation**
  - Routine: issue new → migrate → verify → revoke old
  - Incident: revoke immediately → reissue out-of-band → deploy after access path eliminated
  - Aqua's mistake: rotating through compromised infrastructure

Medium-Term: This Quarter

- **8. GitHub Organization Settings**
  - Organization Settings → Actions → General → Allow select actions
  - Organization Settings → Repository → General → Immutable releases for all repos
- **9. Deploy runtime security monitoring on CI/CD runners**
  - Detect: /proc/*/mem reads, IMDS credential access, encrypted file uploads
- **10. Verify security tooling integrity**
  - cosign + Rekor transparency log for binary verification
  - SLSA provenance attestations for security tool artifacts
  - Consider locally-cached copies instead of pulling from public registries

Medium-Term: This Quarter (continued)

- **11. Monitor for decentralized C2 patterns**
  - Outbound connections to *.icp0.io, *.ic0.app, blockchain API endpoints
  - Systemd user services created by non-interactive processes
  - Unexpected npm publish operations (automated patch bumps)
- **12. Establish a "dependency cool-down" policy**
  - Automated dependency management tools, such as Renovate, can wait for a configurable number of days before upgrading a package

Key Indicators of Compromise

- **Network IOCs:**
  - scan.aquasecurtiy[.]org (45.148.10.212) — Trivy C2
  - checkmarx[.]zone (83.142.209.11) — Checkmarx C2
  - models[.]litellm[.]cloud — LiteLLM C2
  - tdtqy-oyaaa-aaaae-af2dq-cai.raw.icp0.io — ICP blockchain C2
- **File IOCs:**
  - ~/.config/systemd/user/sysmon.py — persistence dropper
  - ~/.local/share/pgmon/service.py — CanisterWorm backdoor
  - /tmp/pglog — downloaded payload (spoofed PostgreSQL)
  - tpcp.tar.gz — encrypted exfiltration bundle
  - litellm_init.pth — malicious .pth file (LiteLLM)
- **GitHub IOCs:**
  - Any repo named tpcp-docs or docs-tpcp — fallback exfiltration

SANS | CLOUD SECURITY

Q&A

10 minutes

SANS | CLOUD SECURITY

Resources & Learn More

SANS CLOUD SECURITY

Threat Intelligence Report

When the Security Scanner Became the Weapon: TeamPCP Supply Chain Campaign

55-Page Detailed Analysis Including

- Complete attack timeline (Feb 28 – Mar 24, 2026)
- MITRE ATT&CK mapping with technique IDs
- Full indicators of compromise (network, file, GitHub)
- Technical payload analysis
- 11 prioritized defensive recommendations
- 30+ cited sources from 14+ security firms

https://www.sans.org/white-papers/when-security-scanner-became-weapon

Key References

- Rami McCarthy — Full TeamPCP Campaign Timeline (ramimac.me/trivy-teampcp)
- CrowdStrike — From Scanner to Stealer: trivy-action Compromise
- Wiz Research — Trivy Compromised by TeamPCP
- Sysdig TRT — TeamPCP Expands: Trivy to Checkmarx
- Aikido Security — CanisterWorm on npm (Charlie Eriksen)
- Dan Lorenc — Six Architectural Failures of GitHub Actions (LinkedIn)
- Chainguard — Imposter Commits in GitHub Actions and CI/CD
- StepSecurity — hackerbot-claw exploitation analysis
- FutureSearch — LiteLLM PyPI supply chain attack
- GitHub Advisory — GHSA-69fq-xp46-6x23 (CVE-2026-33634)

Upcoming SANS Cloud Security Courses

All Cloud Security courses also available OnDemand
sans.org/ondemand

SEC510: Cloud Security Engineering & Controls

Orlando: 3/29 – 4/2	Madrid: 6/1 – 6/5
London: 4/13 – 4/17	Austin: 6/22 – 6/26
San Diego: 5/11 – 5/15	Singapore: 6/6 – 6/10

SEC540: Cloud Native Security & DevSecOps Automation

Orlando: 3/29 – 4/ 2	Amsterdam: 5/18 – 5/22
Denver: 4/20 – 4/24	Chicago: 6/8 – 6/12
San Diego: 5/11 – 5/15	Riyadh: 6/21 – 6/25

SEC588: Cloud Penetration Testing

Orlando: 3/29 – 4/3	Singapore: 6/29 – 7/4
San Diego: 5/11 – 5/16	Washington DC: 7/13 – 7/18
Riyadh: 6/27 – 7/2	Japan: 9/7 – 9/12

FOR509: Enterprise Cloud Forensics & Incident Response

Orlando: 3/29 – 4/3	Munich: 6/8 – 6/13
Denver: 4/20 – 4/25	London: 7/6 – 7/11
San Diego: 5/11 – 5/16	Washington DC: 7/13 – 7/18

SANS

CLOUD SECURITY EXCHANGE SUMMIT 2026

SUMMIT: Aug 17–18 | TRAINING: Aug 19–23

San Francisco, CA & Live Online

IN PARTNERSHIP WITH

aws Google Cloud Microsoft

SANS | CLOUD SECURITY

Thank you for attending!

Kenneth G. Hartman
SANS Certified Instructor
ken@lucid-truth.com

Eric M. Johnson
SANS Fellow
ejohnson@pumasecurity.io